E. Development Tools and Process Setup

Performed by: Dư Hoài Phúc · **Reviewed by:** Ngô Huỳnh Thắm · **Edited by:** None

E.1. Development Process

Performed by: Dư Hoài Phúc · **Reviewed by:** Ngô Huỳnh Thắm · **Edited by:** None

Scrum Methodology

The team adopts **Scrum** as its primary development process, following the course requirements. Scrum provides a structured framework for coordinating work, monitoring progress, and delivering project outcomes incrementally.

Each Programming Assignment (PA) corresponds to one Sprint lasting approximately 2–3 weeks. Every Sprint includes the following activities:

Activity	Frequency	Purpose
Sprint Planning	Once per Sprint	Establish Sprint goals, identify tasks, and assign responsibilities
Scrum Meeting	Twice per Sprint	Report progress, identify obstacles, and coordinate ongoing work
Sprint Review	Once per Sprint	Evaluate completed deliverables and discuss improvements

Meeting records and weekly reports will follow the format specified in the course's *WeeklyReports* document.

Application to the Project

The team uses Jira to organize Sprint tasks and track progress through the following workflow:

```
Backlog → Ready → In Progress → In Review → Done
```

At Sprint Planning, project requirements will be divided into manageable tasks with clearly assigned responsibilities. During the Sprint, members will update their work status and discuss obstacles through Scrum meetings and Messenger. At the Sprint Review, completed deliverables will be evaluated against the Sprint goals.

This approach promotes accountability and provides regular opportunities to identify issues before they affect project delivery.

E.2. Communication and Collaboration Tools

Performed by: Dư Hoài Phúc · **Reviewed by:** Ngô Huỳnh Thắm · **Edited by:** None

The team uses three communication channels, each serving a distinct purpose.

Tool	Purpose	Reason for Use
Moodle	Assignment submission and course information	Official platform designated by the course
Discord	Class-wide announcements, discussions, and questions with the instructor	Provides a common communication channel between students and instructors
Messenger	Internal team communication	Familiar, accessible, and suitable for quick discussions and coordination

Team Communication

Messenger is selected as the primary communication platform among the five team members. It supports daily updates, meeting coordination, technical discussions, and the sharing of urgent information.

Messenger is intended for informal communication rather than formal task tracking. Decisions that affect task ownership, scope, or deadlines must be reflected in Jira so that the project's progress remains traceable.

All members are expected to check team communications regularly and participate actively in project discussions.

E.3. Project and Task Management

Performed by: Dư Hoài Phúc · **Reviewed by:** Ngô Huỳnh Thắm · **Edited by:** None

Selected Tool: Jira

The team selects **Jira** as its primary project management tool because it provides dedicated support for Scrum, Sprint planning, task assignment, workflow management, and progress tracking.

Jira is particularly suitable for this assignment because teamwork performance is evaluated through individual task ownership and documented contributions.

Task Management Rules

Every project-related activity must be recorded in Jira, including:

- Requirement analysis and research.
- Self-training and technical exploration.
- Report writing and documentation.
- Software design and implementation.
- Testing, debugging, and repository maintenance.

Each task must be assigned to **exactly one member**. Although members may collaborate, one person must remain accountable for its completion.

In addition, tasks must be created and assigned **before work begins**. The team must record task creation, assignment, and actual completion dates. A task may only be marked as completed when its work is genuinely finished.

Tasks must not be created and closed retrospectively merely to document work that has already been completed.

Application to Scrum

Jira will be used to maintain the product backlog, organize tasks for each Sprint, and monitor their progress using the following statuses:

```
Backlog → Ready → In Progress → In Review → Done
```

Sprint meetings will use task information from Jira to identify progress, blockers, and remaining work.

Screenshots of all tasks will be included in weekly reports in accordance with the course requirements.

By centralizing task management in Jira, the team aims to maintain transparent responsibilities and reliable evidence of individual contributions.

Evidence of Jira Setup and Task Management

The following screenshots demonstrate the Jira project configuration and task-tracking activities.

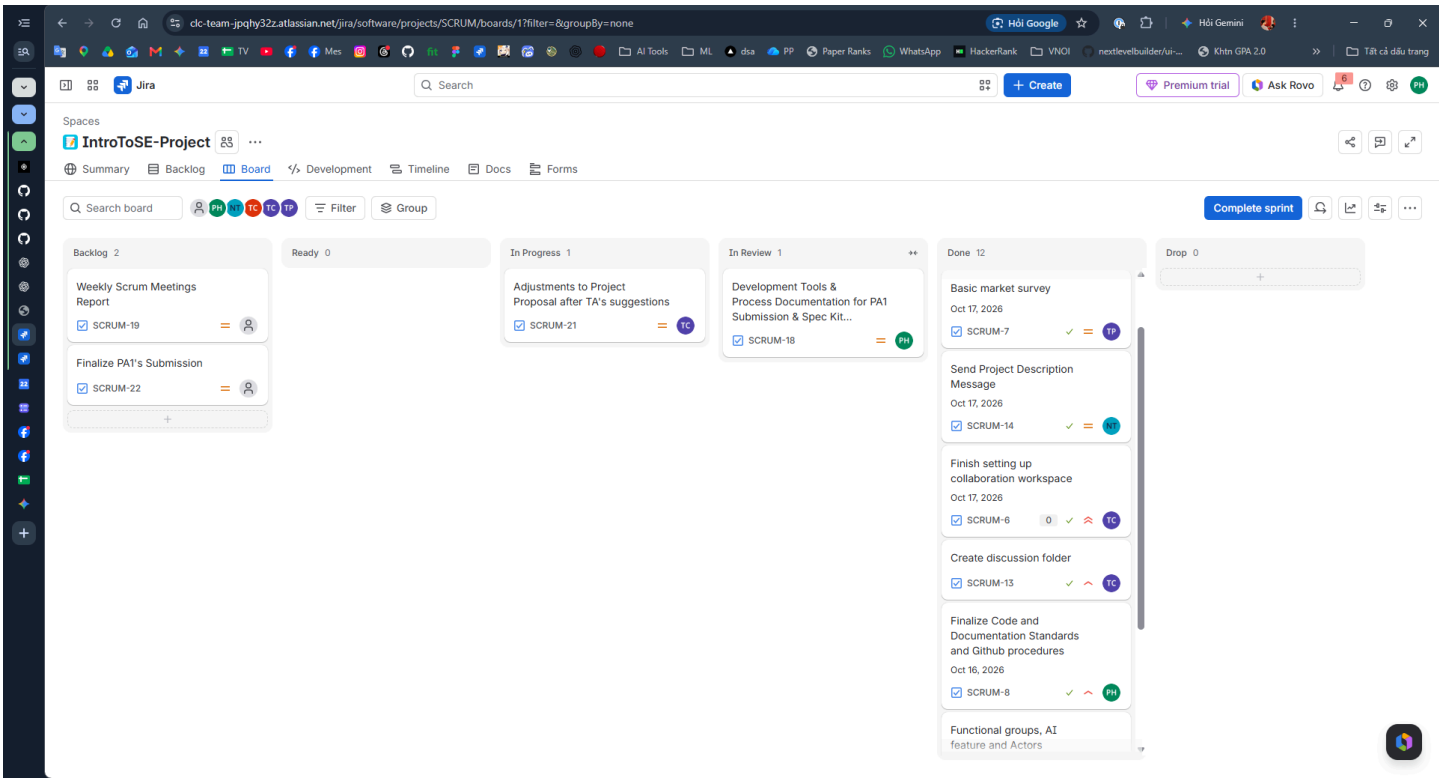


Figure E.1. Jira Scrum Board showing the team's task management workflow.

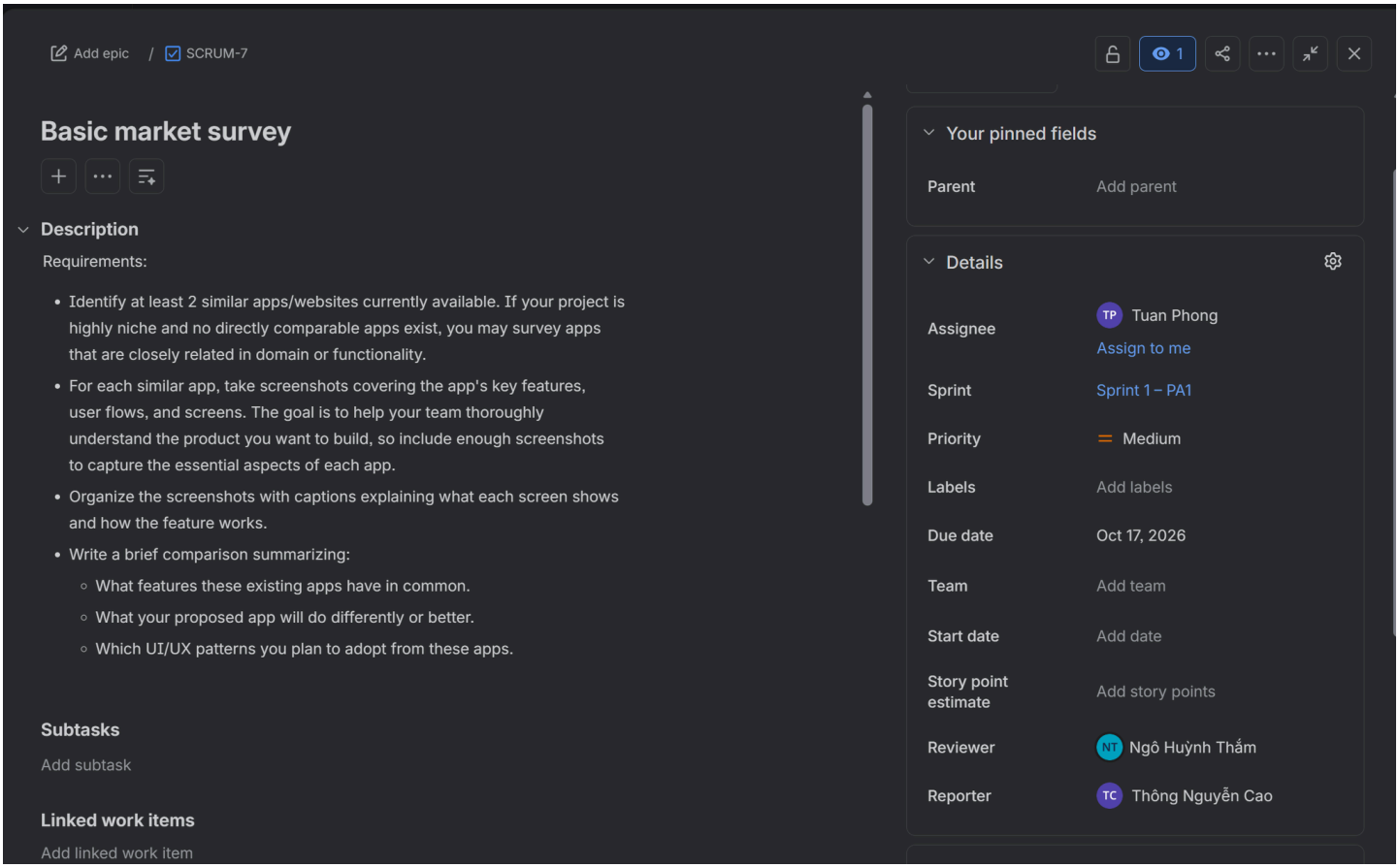


Figure E.2. Jira issue details showing task ownership, status, and recorded activity.

E.4. Version Control and Code Collaboration

Performed by: Dư Hoài Phúc · **Reviewed by:** Ngô Huỳnh Thắm · **Edited by:** None

Selected Tools: Git and GitHub

The team uses **Git and GitHub** for version control and collaborative development.

GitHub was selected because it provides a unified environment for managing source code, Markdown documentation, feature branches, Pull Requests, code reviews, and automated checks.

A private repository has been established for the project:

CSC13002-IntroToSE-Project

The repository stores documentation and development configurations alongside application code as it is introduced.

GitHub Flow

The team adopts **GitHub Flow** to keep development simple and organized. The intended collaboration process is:

1. Select an assigned Jira task.
2. Create a feature branch from `main`.
3. Implement changes and perform local verification.
4. Open a GitHub Pull Request referencing the Jira Issue Key.
5. Complete automated checks and peer review.
6. Squash Merge approved changes into `main`.
7. Update the corresponding Jira task.

Branch names include the Jira Issue Key, while commit messages follow **Conventional Commits**.

For example:

```
Branch: feat/SCRUM-12-fridge-inventory
Commit: feat(web): add fridge inventory page SCRUM-12
```

The team aims to obtain at least one peer approval before merging. Changes affecting authentication, database schemas, or shared interfaces may require additional review.

Documentation and Contribution Tracking

Project documentation is maintained in Markdown and version-controlled through GitHub. This allows changes to requirements, design documents, and project reports to be reviewed alongside source code.

The repository also includes a documentation attribution mechanism to record the original performer, subsequent editors, and reviewers of academic documents.

This mechanism is intended to complement GitHub's contribution history and support transparent documentation of individual work.

For security, API keys, credentials, and other secrets must never be committed to the repository.

Evidence of GitHub Repository Setup

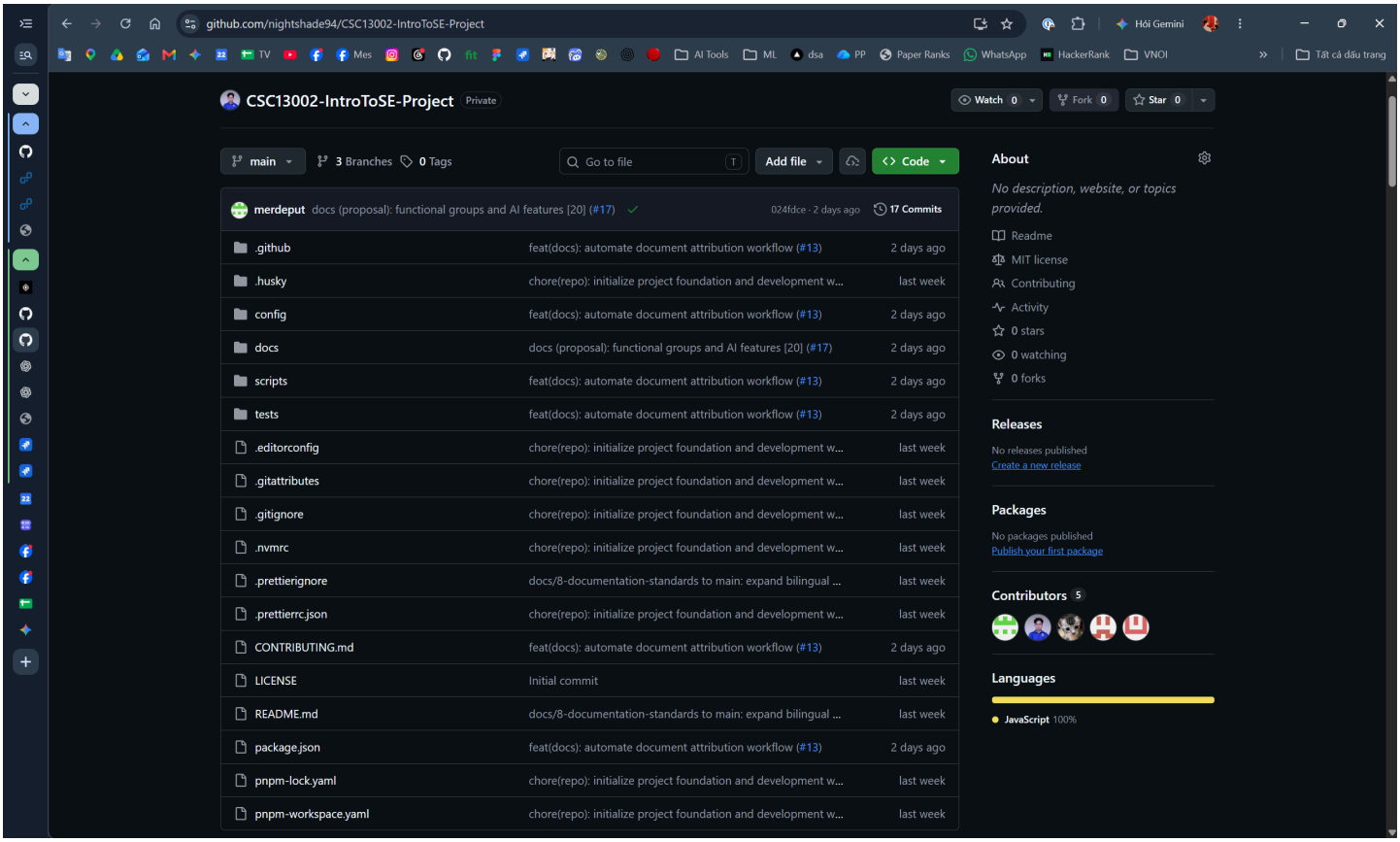


Figure E.3. Private GitHub repository structure containing project documentation and development files.

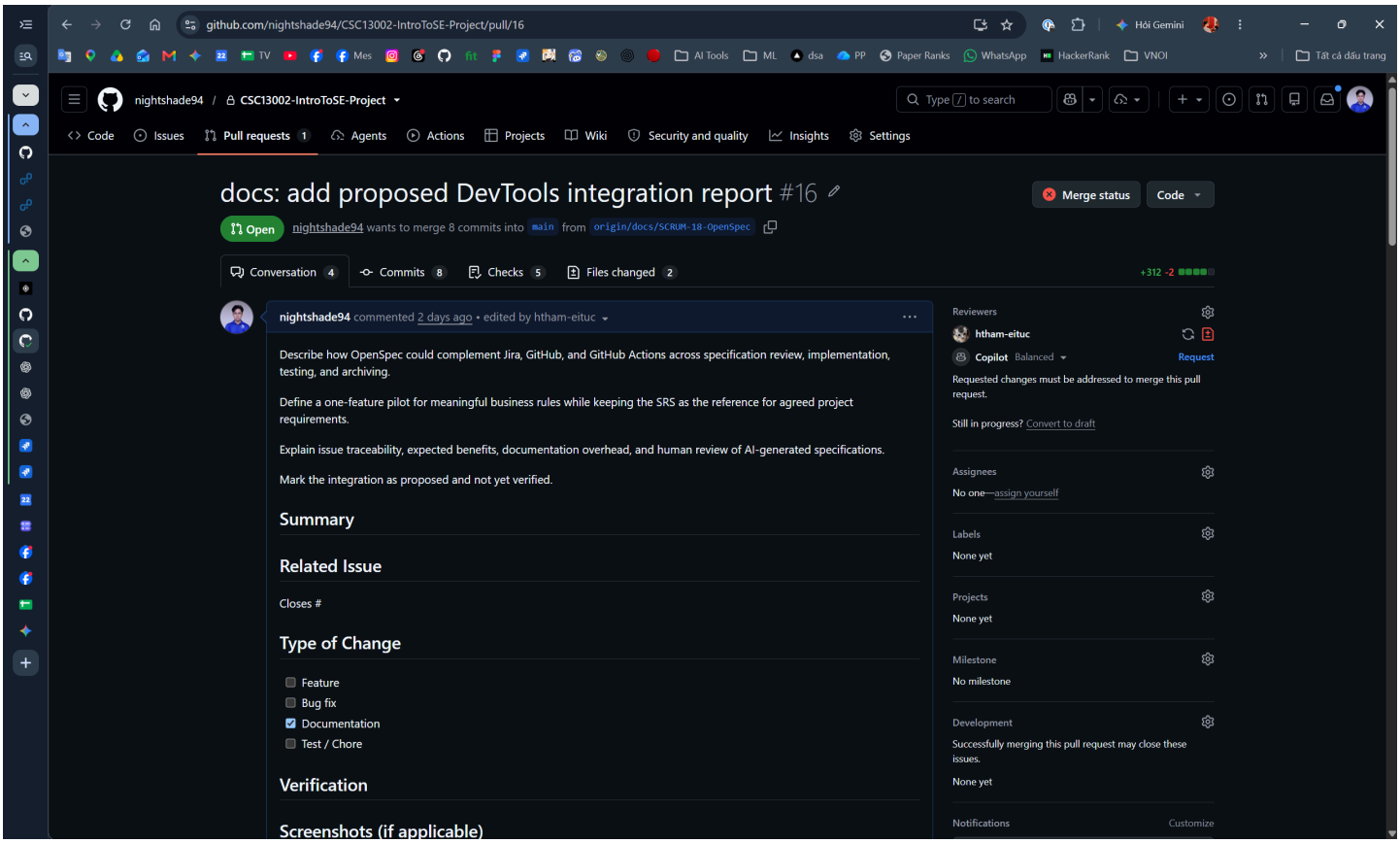


Figure E.4. GitHub Pull Request used for collaborative development and code review.

E.5. Development Environment and Quality Assurance

Performed by: Dư Hoài Phúc · **Reviewed by:** Ngô Huỳnh Thắm · **Edited by:** None

Development Toolchain

The team has established a shared development foundation to support consistent workflows across different development environments.

Tool	Role	Status
Node.js 24 LTS	Standardized JavaScript runtime	Established
pnpm	Dependency and workspace management	Established
Prettier	Consistent formatting	Established
Husky + lint-staged	Lightweight pre-commit validation	Established
GitHub Actions	Automated repository checks	Established
ESLint	Static code analysis	Planned

Tool	Role	Status
Vitest	Automated unit testing	Planned
React Testing Library	Frontend component testing	Planned
Supertest	Backend API testing	Planned

These tools were selected for their compatibility with the intended JavaScript and TypeScript technology stack and their ability to support consistent development practices.

Application Architecture

The planned application follows a monorepo structure with:

- **Frontend:** React, Vite, and TypeScript.
- **Backend:** Node.js, Express, and TypeScript.
- **Workspace Management:** pnpm.

This structure is intended to simplify dependency management and support shared development conventions across frontend and backend components.

Quality Assurance

The repository currently supports formatting checks, lightweight Git hooks, and automated repository validation through GitHub Actions.

As application development progresses, the team intends to expand CI to include linting, type checking, automated tests, and build verification.

Pull Requests will serve as the main integration checkpoint, allowing members to review changes and identify problems before merging.

The objective is to introduce quality assurance practices incrementally without imposing unnecessary complexity during the early project stages.

Evidence of Continuous Integration

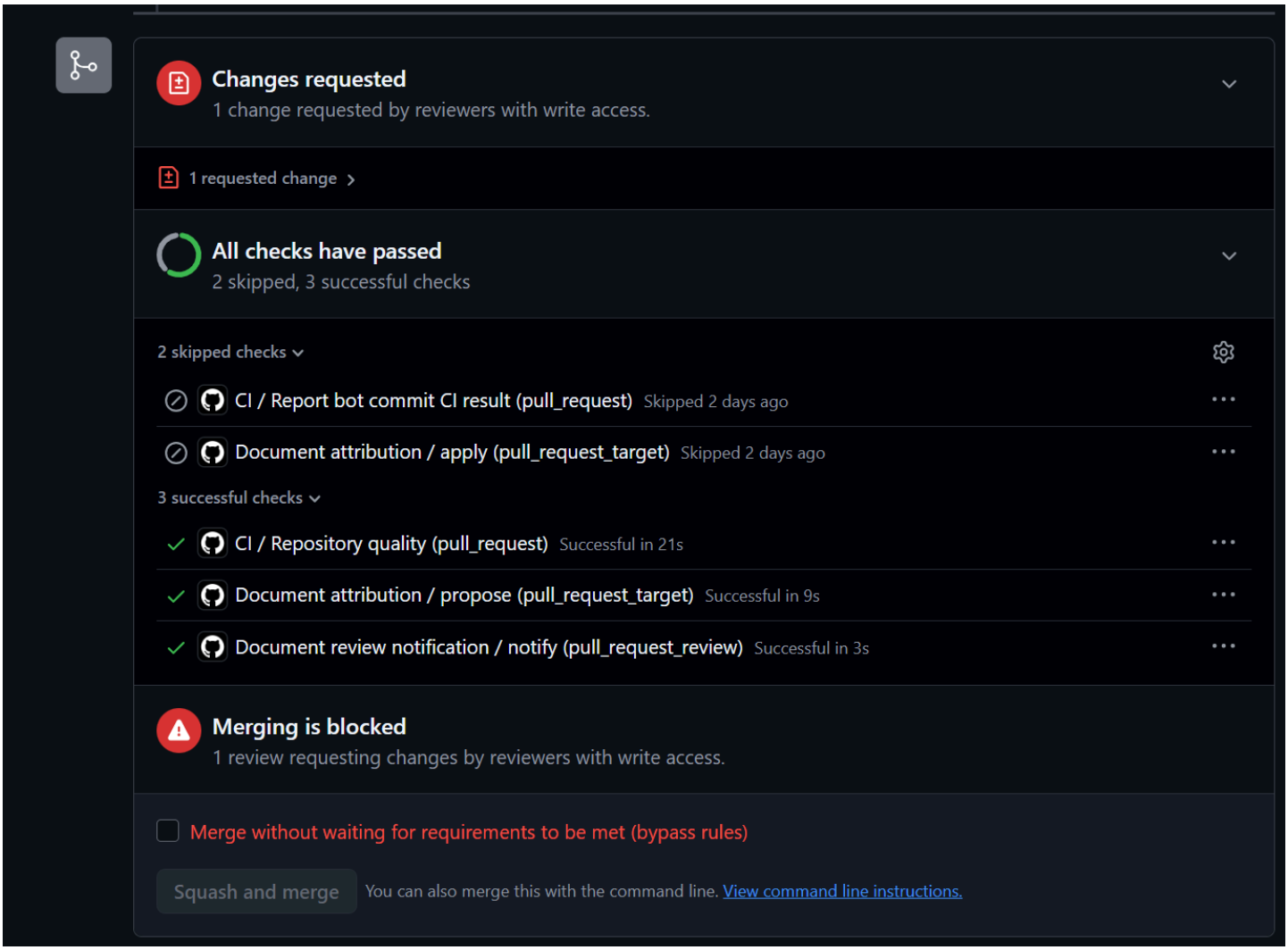


Figure E.5. GitHub Actions workflow execution for repository quality checks.

E.6. Specification-Driven and AI-Assisted Development

Performed by: Dư Hoài Phúc · **Reviewed by:** Ngô Huỳnh Thắm · **Edited by:** None

Specification-Driven Development

Specification-Driven Development (SDD) emphasizes defining system requirements, expected behavior, and implementation plans before writing code.

This approach is particularly relevant to Smart Fridge Planner because the project involves multiple functional areas, such as food inventory management, expiry tracking, recipe recommendations, and household collaboration.

Clear specifications can help reduce ambiguity, align team members, and provide structured guidance for AI-assisted implementation.

Evaluation of Specification Tools

The team considered three approaches for managing specifications.

Tool / Approach	Advantages	Limitations
Manual Specifications	Flexible, familiar, and requires no additional tools	Greater effort to keep requirements and implementation synchronized
GitHub Spec Kit	Structured specification-to-implementation workflow and alignment with the course's required tooling	Additional process and documentation overhead for a small team
OpenSpec	Lightweight, iterative, change-oriented, and compatible with multiple AI coding assistants	Requires learning its workflow and maintaining specifications consistently

Selected Tool: OpenSpec

After evaluating these approaches, the team selected **OpenSpec** as its preferred specification-driven development solution.

OpenSpec was favored for several reasons:

Lightweight workflow. OpenSpec provides a structured way to describe software changes without requiring an overly rigid development process. This is suitable for a five-member student team.

Iterative specification management. Requirements and technical plans can be refined as the team gains a clearer understanding of the project.

AI-assisted development support. OpenSpec organizes feature requirements, scenarios, technical decisions, and implementation tasks into artifacts that can guide AI coding assistants.

Change traceability. Each significant feature can be associated with a proposal, specifications, design considerations, and implementation tasks, improving the connection between planning and delivery.

Compatibility with existing tools. OpenSpec can operate alongside Jira for task management and GitHub for version control without replacing either system.

Planned Integration

The team intends to introduce OpenSpec into its development workflow as follows:

1. **Jira:** Record, prioritize, and assign a feature task.
2. **OpenSpec:** Define the proposed change, requirements, design, and implementation tasks.
3. **Team Review:** Verify that the specification reflects the intended functionality.
4. **AI-Assisted Implementation:** Develop the feature using the reviewed specification as guidance.
5. **GitHub:** Submit the implementation through a Pull Request.
6. **Verification:** Review the implementation and run the available automated checks.
7. **Specification Update:** Archive the completed change and maintain the current specifications.

OpenSpec will primarily be used for significant features and architectural changes. Minor fixes or documentation updates may follow the standard Jira and GitHub workflow without requiring a separate specification.

This selective approach is intended to reduce unnecessary documentation overhead.

Course Requirement Consideration

The course instructions explicitly require **GitHub Spec Kit** setup and initialization starting from PA2.

Although OpenSpec is the team's preferred solution, its use as a substitute for Spec Kit requires confirmation from the instructor.

If substitution is not permitted, the team will fulfill the mandatory Spec Kit requirements while avoiding unnecessary duplication between specification workflows.

E.7. AI Coding Tools

Performed by: Dư Hoài Phúc · **Reviewed by:** Ngô Huỳnh Thắm · **Edited by:** None

Purpose

AI coding assistants are incorporated into the development workflow to support productivity, technical learning, and software quality. They can assist team members with understanding unfamiliar technologies, generating implementation suggestions, identifying errors, writing tests, and improving documentation.

All five team members have registered for AI coding accounts, as required by the course. However, the specific tools used may differ depending on individual preferences and development needs.

Team Members and Selected Tools

Team Member	AI Coding Tool(s)	Main Usage
Nguyễn Cao Thông	ChatGPT Plus + Codex	Code, Documentation and Review

Team Member	AI Coding Tool(s)	Main Usage
Ngô Huỳnh Thắm	ChatGPT Plus + Codex	Code, Documentation and Review
Phan Tuấn Phong	ChatGPT Plus + Codex	Code, Documentation and Review
Dư Hoài Phúc	ChatGPT Plus + Codex	Code, Documentation and Review
Cao Tiến Thiên	Claude	Code, Documentation and Review

Application in the Development Workflow

AI coding assistants may be used to support the following activities:

- **Requirement Analysis:** Clarifying requirements and exploring possible solutions.
- **Implementation:** Suggesting code structures and implementation approaches based on agreed specifications.
- **Debugging:** Explaining errors and suggesting potential fixes.
- **Testing:** Assisting with test case identification and automated test generation.
- **Documentation:** Improving technical explanations and maintaining development documents.
- **Self-Training:** Helping members understand unfamiliar frameworks, libraries, and development practices.

Where applicable, AI assistants will use reviewed OpenSpec artifacts as context to keep implementation aligned with the intended requirements.

Responsible AI Usage

AI-generated results are treated as suggestions rather than verified solutions. Each team member remains responsible for understanding, reviewing, testing, and validating the work submitted under their assigned Jira tasks.

The team will also avoid sharing credentials, API keys, private user information, or other sensitive data with AI tools.

AI Coding Assistants

All five team members have registered for AI coding accounts.

AI assistants may support requirement analysis, technical exploration, implementation, debugging, testing, and documentation.

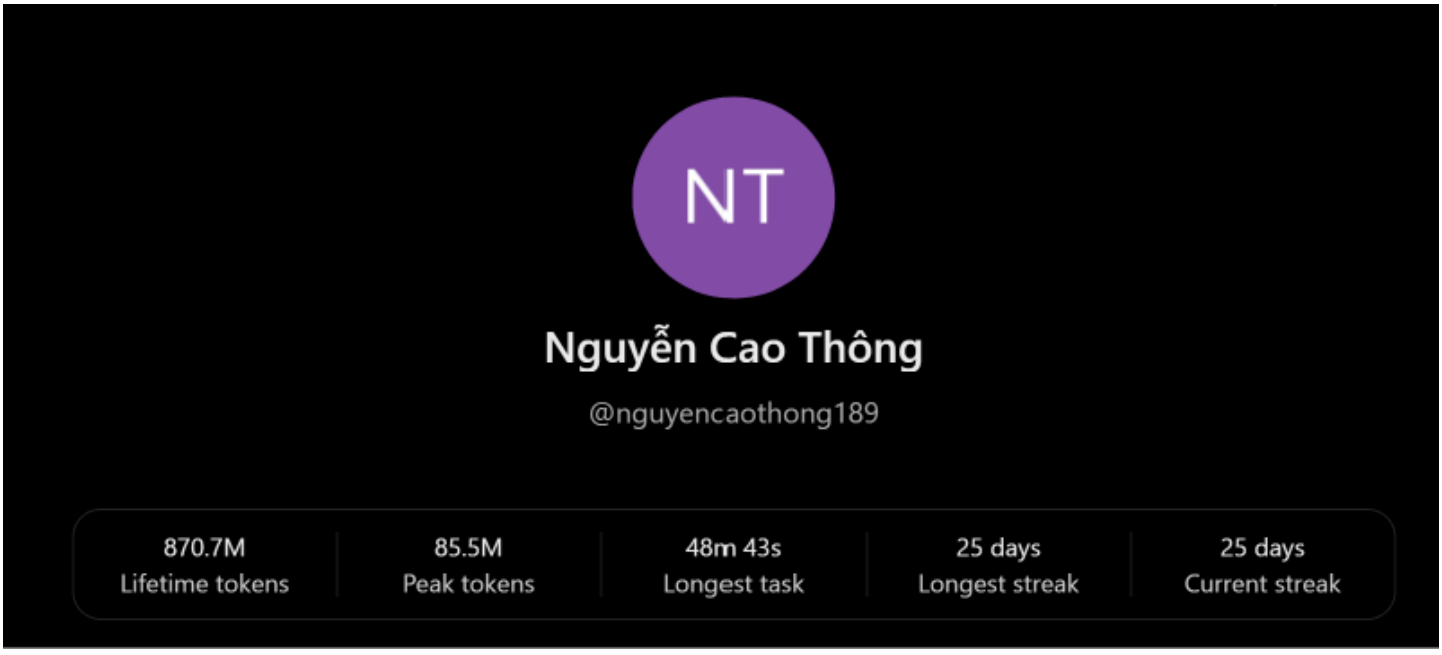
However, AI-generated outputs must be reviewed and verified by the responsible team members. The use of AI does not replace individual accountability for assigned tasks or the quality of submitted deliverables.

Evidence of AI Coding Accounts

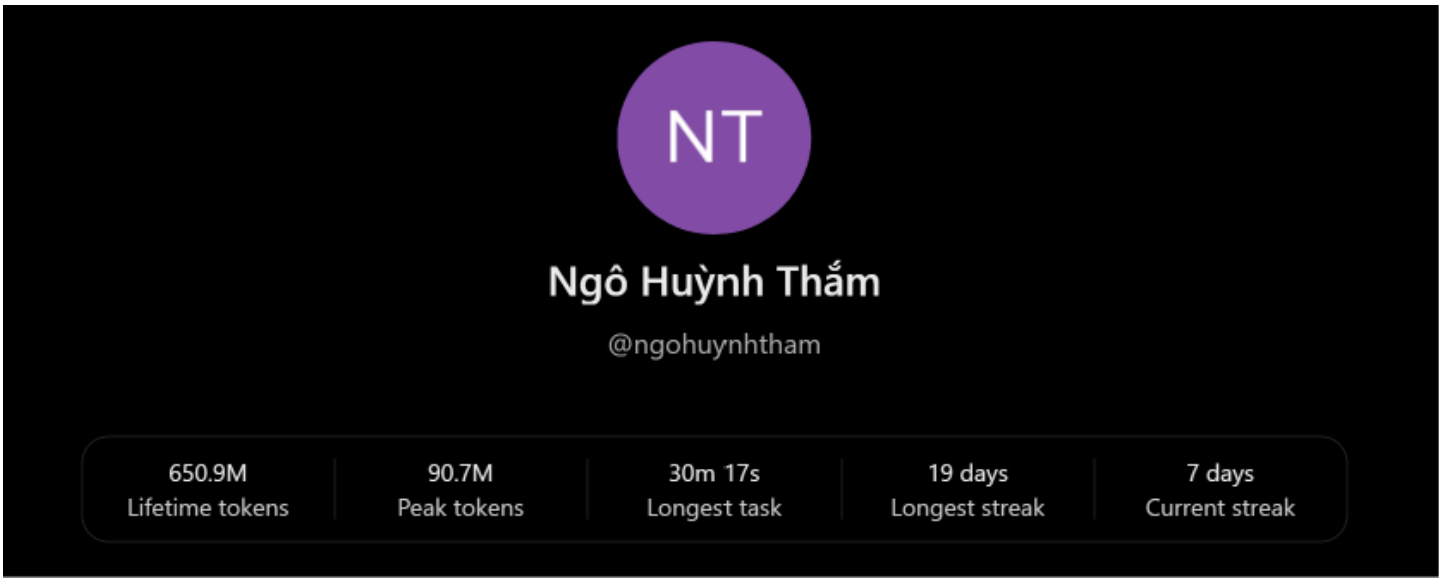
The following screenshots provide evidence that all five team members have registered for AI coding accounts.



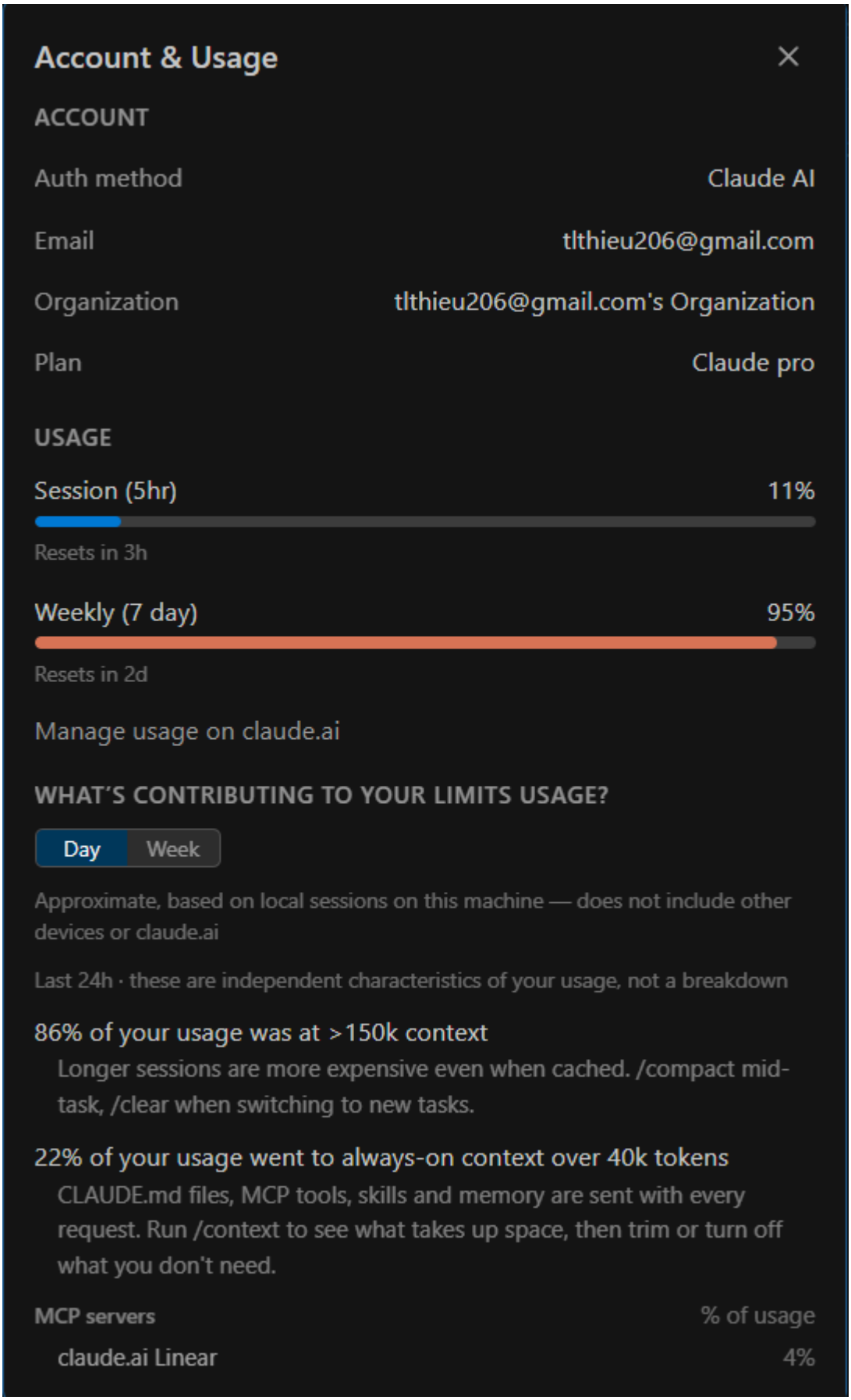
AI coding account of Dư Hoài Phúc.



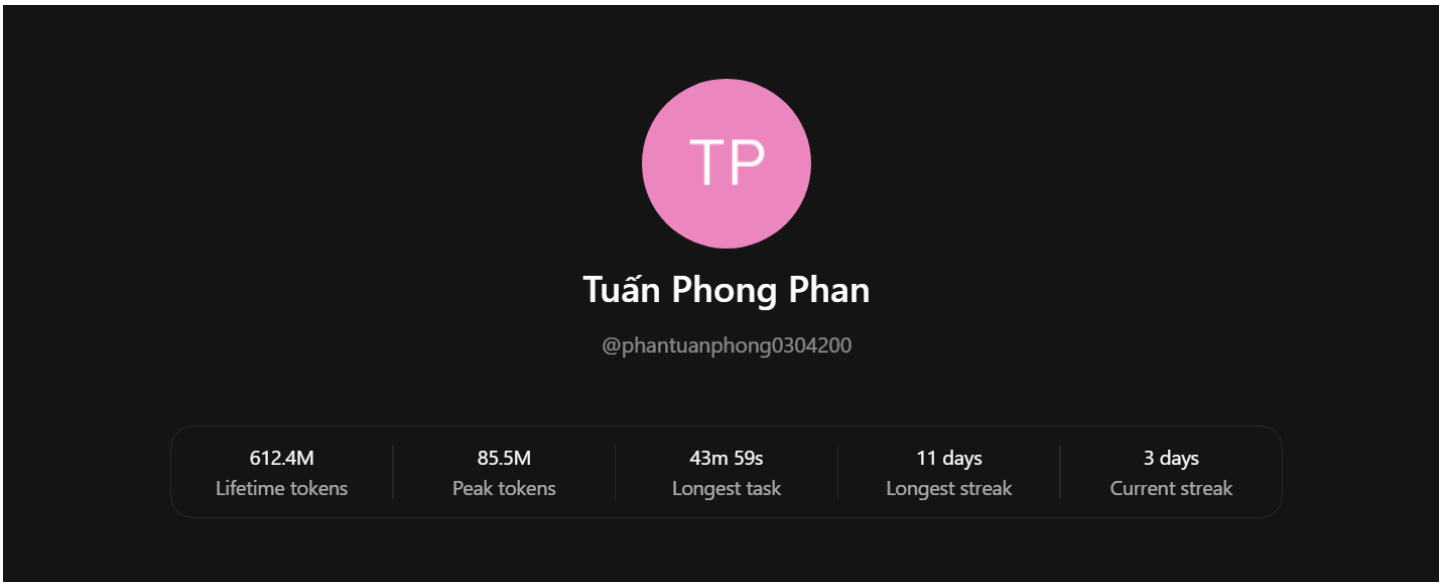
AI coding account of Nguyễn Cao Thông.



AI coding account of Ngô Huỳnh Thắm.



AI coding account of Cao Tiến Thiên.



AI coding account of Phan Tuấn Phong.

Conclusion

AI coding assistants are intended to improve development efficiency and support technical learning. Their use complements, rather than replaces, individual responsibility, teamwork, code review, and software engineering practices.

E.8. Overall Workflow Integration

Performed by: Dư Hoài Phúc · **Reviewed by:** Ngô Huỳnh Thắm · **Edited by:** None

The selected tools have complementary responsibilities within the project.

Development Activity	Primary Tool
Assignment submission	Moodle
Course communication	Discord
Internal communication	Messenger
Scrum planning and task management	Jira
Specification-driven development	OpenSpec (preferred; subject to course approval)
AI-assisted development	AI Coding Assistants
Version control and code review	GitHub
Automated quality checks	GitHub Actions

The intended overall workflow is:

Sprint Planning → Jira Task Assignment → Specification and Design → Implementation → GitHub Pull Request → CI and Peer Review → Sprint Review

Jira provides visibility into task ownership and progress, OpenSpec provides structured feature specifications, and GitHub maintains the history of implementation and review activities.

This separation of responsibilities is intended to support traceability without duplicating the primary functions of each tool.

E.9. Conclusion

Performed by: Dư Hoài Phúc · **Reviewed by:** Ngô Huỳnh Thắm · **Edited by:** None

The Smart Fridge Planner team has selected a development environment that emphasizes collaboration, accountability, and practical Software Engineering practices.

Scrum provides the overall development process, Jira supports Sprint and task management, Messenger enables daily communication, and GitHub supports collaborative version control and code review. Pre-commit tooling and GitHub Actions contribute to development consistency and quality assurance.

After comparing specification management approaches, the team selected **OpenSpec** as its preferred SDD tool because of its lightweight and iterative workflow, suitability for AI-assisted development, and compatibility with the team's existing tools. Its use as a substitute for the course-required Spec Kit remains subject to instructor approval.

Together, these tools are intended to establish a **structured, manageable, and traceable development workflow** that supports both project delivery and the educational objectives of the Introduction to Software Engineering course.